International Journal on Software Tools for Technology Transfer (2025) 27:313–332
https://doi.org/10.1007/s10009-025-00802-4

GENERAL

Special Issue: FMICS 2023

Pattern-based verification of ROS 2 applications using UPPAAL

Lukas Dust[1] · Rong Gu[1] · Cristina Seceleanu[1] · Mikael Ekström[1] · Saad Mubeen[1]

Accepted: 3 April 2025 / Published online: 23 April 2025

Abstract

This paper proposes an approach to pattern-based modeling and UPPAAL-based verification for ROS 2 applications. The proposed verification focuses on callback execution latencies and buffer overflow. We propose formal model templates to model the execution of ROS 2 system components, created using a pattern-based approach. The model templates simplify the formal modeling of an ROS 2 application. Using UPPAAL, we model in UPPAAL timed automata, allowing the description of computation chains of ROS 2-based applications. Our focus is on execution behavior, including two versions of the mainline single-threaded executor of ROS 2. System traces generated using the formal models are validated in multiple experiments. Furthermore, we compare two approaches to modeling the execution of nodes that are typically the core units of computation of ROS 2. The first approach is a holistic approach to model ROS 2 applications, including communication and execution in computation chains. The second is an approach for individual nodes only, at a higher abstraction level. Additionally, we show the application of the verification by model checking in two ROS 2 system scenarios where we compare generated model traces to actual system executions. Overall, through formal modeling and verification, we showcase the potential for uncovering errors in the execution of distributed robotic systems.

Keywords Robot operating system 2 · Pattern-based modeling · Model checking · Uppaal

1 Introduction

Robotic systems frequently carry significant safety implications and, therefore, have stringent timing constraints. Their computational processes are commonly distributed across various actors and communication networks. As these systems are composed of distributed elements, ensuring the correctness of system design and implementation presents an enormous challenge for robotics developers.

The open-source middleware Robot Operating System (ROS) [16] has been introduced in an attempt to standardize the design of distributed robotic systems, focusing on rapid prototyping of robotic applications. Following the emerging needs for robotics, ROS 2 [15] has been released to update ROS by adding a real-time capable communication approach. As ROS 2 tackles modern needs, the support for ROS will end in 2025 [13]. While both versions of ROS are widely used in academia and industry [2], users of ROS have to update their systems to ROS 2. Analytical methods and interfaces for ROS have been created and are well understood, but ROS 2 theoretical analysis and interfacing tools still need to catch up. Moreover, the potential problems arising from transitioning between ROS and ROS 2, including changes in execution behavior, have not been thoroughly examined. Existing methods of response-time analysis of processing chains [6, 8] and end-to-end timing analysis [19] require manual and intensive computation that varies from system to system. The complexity brought by many system configuration options, such as communication buffer sizes, complicates analyzing these systems. Hence, automated tool support is essential.

✉ L. Dust
lukas.dust@mdu.se

R. Gu
rong.gu@mdu.se

C. Seceleanu
cristina.seceleanu@mdu.se

M. Ekström
mikael.ekstrom@mdu.se

S. Mubeen
saad.mubeen@mdu.se

[1] Mälardalen University, Västerås, Sweden

The multiple versions of ROS 2 [14] and a lack of documentation bring more obstacles to the analysis of ROS 2 systems. An example of a problem with constant evolution and need for documentation is the existence of various scheduling mechanisms, which are conducted by *executors*. An executor conducts the scheduling of time or data-triggered functions (aka, *callbacks*) inside main components of an ROS 2 system, namely *nodes*. The executor underwent an update, reflecting a semantic change between ROS 2 versions, specifically from executor version one (ExV1) in *Dashing* to executor version two (ExV2) in *Eloquent* [6]. Consequently, different scheduling behavior can be observed depending on the version used in the implementation. Changes in scheduling have been discovered, but not sufficiently regarded in the documentation. In other words, the execution of callbacks may exhibit unexpected behavior such as buffer overflow, causing *instance misses* and unexpected *latency* of callbacks. Specifically, instance misses mean that instances of callback execution are skipped and thus never appear in the system execution. The latency of callbacks describes the time from the release of a callback until the end of its execution.

The lack of analytical methods and tools makes the trial-and-error approach dominant in the verification and testing of robotic systems [17]. In contrast, formal methods, such as *model checking* [5], can provide rigorous analysis of systems. With model checking, exhaustive verification of a system model against its requirement specification is made possible by automatically exploring the state space of the system model. Nevertheless, model checking requires formal models of systems, which are complicated to create, especially for complex and distributed systems, like ROS 2-based ones.

Previously, we have proposed an approach to simplify the process of formally modeling ROS 2 nodes [10]. Our solution relies on a pattern-based verification of input buffer sizes and callback latency for ROS 2 nodes. We have employed *Timed automata* (TA) [3] as the formal modeling language, and UPPAAL [12] as the model checker. To ease modeling of ROS 2 nodes, we have implemented TA templates for ROS 2 executors and callbacks, which can be initialized with parameters only, upon reuse, without modifying the TA templates. We demonstrate the applicability of our approach in an experimental evaluation. Bugs in our benchmark ROS 2 system can be found both in the experiments and our model-checking approach. Furthermore, we show that the model-checking approach can reveal potential errors of system design that are not detected during a long period of experiments, which shows the advantage of our approach against the trial-and-error methods.

However, our model-checking approach is strictly limited to modeling nodes individually. This paper presents improvements to the TA templates, as well as proposes new TA templates that enable verification of multiple executors and holistic ROS 2 applications. Specifically, we adapt the TA templates of callbacks and their executor such that callbacks can be assigned to specific executors now. Furthermore, in our previous work, communication has been abstracted as release times of callbacks and timers, whereas in this paper, we create TA templates for the modeling of communication between callbacks, making modeling more realistic and providing grounds for a more in-depth verification. In particular, we design TA templates for communication channels, and adapt callback templates to employ them for communication. Additionally, the templates of the communication channels now include a model of output buffers of ROS 2 nodes, whose sizes can be verified. We propose general improvements to our previous TA templates, such as adding new parameters for offsets of time-triggered callbacks. Furthermore, an improved handling of time in the executor templates allows a reduction of the state space when model checking, which decreases the verification time. The improvements also allow for specifying different prioritization of system components, which can be used for modeling various assumptions of the system under verification. Furthermore, the new approach can generate counterexamples as traces that reveal the critical system execution paths, which we have not shown in the previous approach (e.g., for longest latency). Our evaluation in this paper compares the model-checking approaches in our previous paper [10] with the new approach that supports verifying holistic systems. We validate the models experimentally by showing that observed systems behavior in experiments can be exhibited using our model-checking approach. Furthermore, we show how different assumptions can lead to the detection of potential system errors that can serve as warnings to developers.

To summarize, in this paper, we answer the following research questions. **RQ1**: Given the two versions ExV1 and ExV2 of single-threaded executors in ROS 2, how to ensure the correctness of a design of ROS 2-based systems with respect to the behavior of timer callbacks, callback latency, and the sizes of input buffers of callbacks? **RQ2**: What are the patterns for modeling ROS 2 systems, which can be reused in verification? **RQ3**: Can model checking mirror the behavior of ROS 2 systems, as shown in experiments, but also reveal potential errors that are not discovered via experimental evaluation? **RQ4**: How does the approach of modeling an application holistically differ from the approach of modeling callbacks individually?

This work is an extension of our previous conference paper [10] published in the Formal Methods for Industrial Critical Systems, FMICS 2023, with the following new contributions:

- We improve the TA templates by adding parameters, such as offset, to the time-triggered callbacks and time handling to reduce the state space generated by model checking.

- We adapt the TA templates to assign callbacks to specific executors, allowing verification of multiple executors simultaneously. Hence, verification of a ROS 2 system can be done holistically.
- We add the ability to model communication in a ROS 2 system, achieved by adding TA templates for modeling ROS 2 communication channels and data-triggered callbacks. Furthermore, we give callback templates the ability to utilize the communication channels to simulate data sending.
- We include model checking of output buffer sizes of callbacks, besides those of input buffers, and merge variables of the previous TA templates.
- We evaluate and compare the previous approaches of modeling nodes individually to the new approach of modeling an ROS 2 system holistically.

The remainder of the paper is structured as follows. Section 2 presents the concepts of ROS 2, TA, and UPPAAL. In Sect. 3, we present the feature abstractions, which the model templates are based on. In Sect. 4, we introduce the new model templates, followed by a demonstration of the application of verification and modeling in Sect. 5. We evaluate the proposed model templates on various examples, in Sect. 6. Based on the results, we show the answers to the research questions, in Sect. 7. In Sect. 8, we give an overview of the related work before concluding the paper with final remarks in Sect. 9.

2 Background

This section overviews the preliminaries for the rest of the paper, by recalling the main concepts of ROS 2 first, including the internal scheduling mechanism, followed by a brief description of Timed Automata and UPPAAL.

2.1 ROS 2

ROS 2 is an open-source software development kit for robotics applications. The purpose of ROS 2 is to offer a standard software platform to developers across industries, being updated continuously, as various stable distributions.

ROS 2 system model Conceptually, the software of an ROS 2-based system relies on a graph of connected endpoints, called *nodes*, which are communicating with each other by sending and receiving messages. An example of such a system is given in Fig. 1, where two nodes, represented by green ellipses, communicate with each other over defined communication channels of ROS 2. The main patterns of communication are *publisher–subscriber* and *service–client* communication. The publisher–subscriber pattern is a one-to-many communication model, where a node (*publisher*) sends messages to a specific communication channel called *topic*. Other nodes (*subscribers*) receive those messages by subscribing to the same topic. The publisher and the subscriber do not need to know each other's identity, location, or existence. They only need to agree on the name and the format of the topic. This allows for loose coupling and high flexibility. The event of incoming data in the receiving node triggers a subscription-specific function called *subscription callback*. In Fig. 1, there are four topics represented by the red boxes (`Control, H, M, L`). **Node 0** publishes data while **Node 1** receives the data, depicted by the direction of the arrows. Hence, **Node 0** contains the publishers and **Node 1** contains the subscriptions. Note that for simplicity the functions (callbacks) are not shown. The second pattern of communication is the service–client communication. This kind of communication is a request–response communication model, where a node (*client*) sends a request to another node (*service*) and waits for a response. The client and the service need to know each other's name and the format of the request and response messages. They also need to establish a connection before exchanging data. This allows for synchronous and reliable communication.

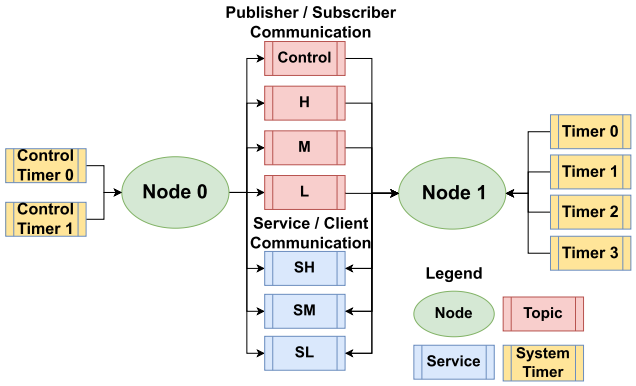


Fig. 1 Schematic example of an ROS 2 system containing two nodes communicating with each other (Color figure online)

In Fig. 1, service–client channels are marked in blue. In the given example, a directed request is sent from the requesting node (**Node 0**) to the receiving node (**Node 1**). Consequently, a *service callback* starts executing in **Node 1**. In response to the *service callback*, a *client callback* on arrival in the requesting **Node 0** is triggered. Every registered access to a communication channel where data is received (Subscriber, Service, Client) has an individual FIFO input buffer, whose size is configurable. Besides communication, there is another method of triggering callbacks in ROS 2, via the system *timers*, denoted in yellow in Fig. 1. Timers define *time-triggered callbacks*, which access data in the node or receive data from an external interface. Timers can be configured as periodic or sporadic, in the latter case timer callbacks

being triggered several times sporadically (i.e., when given a set of wall times).

ROS 2 scheduling and execution The execution of ROS 2 relies on a host operating system that assigns resources to each node that executes the ROS 2 internal scheduler module, called *the executor*. The executor's task is to schedule the execution order of callbacks in a node, each of them being one of the four types of callbacks (subscriber, service, client, timer) mentioned above. The ROS 2 executor stores all timers, subscriptions, clients, and services of all registered nodes, updating their state during the schedule. A callback function can be seen as a recurrent task and each execution of a callback function is a task instance (or job). The default executor operates on a single thread in the host operating system. The scheduling performed by the executor is non-preemptive, meaning that a callback cannot interrupt an other callback's execution. Furthermore, the scheduling is set-based and polling-point-based, meaning that when the executor is idle, a set (`ready set`) with one instance of each released callback at the given point in time (`polling point`) is created. The following scheduling decisions are then made based on the callbacks contained in the set. After each execution, the executed callback is removed from the ready set. The next ready set is created after each callback in the previous ready set has been executed once and the executor is idle. While ROS 2 has evolved, the executor has changed so that there exist two versions of the executor (ExV1 [8] and ExV2 [6]). In ExV1, timer callbacks are excluded from the ready set. Hence, expired timers are considered for scheduling in each scheduling instance, while the other types of callback are only considered for scheduling if they are included in the ready set. In ExV2, timers are included in the ready set. Hence, only timers expired before the polling point are considered for scheduling until the set has been emptied and the next polling point occurs.

2.2 Timed automata and UPPAAL

Since our formal verification relies on the Timed Automata (TA) language, in this subsection, we recall the formal definitions of TA and its semantics, respectively, and we overview briefly the TA-based model checker Uppaal that we employ in this paper. For more details, the reader is referred to the relevant literature [3, 12].

Definition 1 ([3])
A Timed Automaton (TA) is a tuple

$$\mathcal{A} =< L, l_0, C, \Sigma, E, Inv >, \tag{1}$$

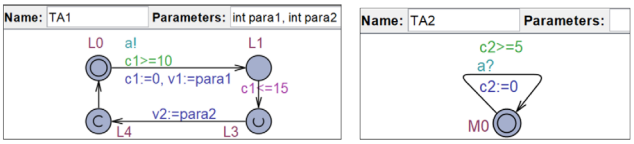
where L is a finite set of locations, l_0 is the initial location, C is a finite set of nonnegative real-valued variables called *clocks*, Σ is a finite set of actions, $E \subseteq L \times \mathcal{B}(C) \times \Sigma \times 2^C \times L$ is a finite set of edges, where $\mathcal{B}(C)$ is the set of *guards* over C, that is, conjunctive formulas of constraints of the form $c_1 \bowtie n$ or $c_1 - c_2 \bowtie n$, where $c_1, c_2 \in C$, $n \in \mathbb{N}$, $\bowtie \in \{<, \leq, =, \geq, >\}$, 2^C is a set of clocks in C that are reset on the edge, and $Inv : L \rightarrow \mathcal{B}(C)$ is a partial function assigning invariants to locations.

Definition 2
Let $< L, l_0, C, \Sigma, E, Inv >$ be a TA. Its semantics is defined as a *labeled transition system* $< S, s_0, \rightarrow >$, where $S \in L \times \mathbb{R}^C$ is a set of states denoted as (l, u) where l means the location and u means the evaluation of clocks in C, $s_0 = (l_0, u_0)$ is the initial state, and $\rightarrow \subseteq S \times (\mathbb{R}_{\geq 0} \cup \Sigma) \times S$ such that:

1. (delay transition) $(l, u) \xrightarrow{d} (l, u \oplus d)$, where $d \in \mathbb{R}_{\geq 0}$, and $u \oplus d$ increases the value of clocks in C by d such that $\forall d' \leq d, u \oplus d' \models Inv(l)$ meaning that $Inv(l)$ is true when the evaluation of clocks is $u \oplus d'$, and
2. (action transition) $(l, u) \xrightarrow{a} (l', u')$, if there exists $e = (l, g, a, r, l') \in E$ such that $u \in g$, $u' = [r \mapsto 0]u$ is a new evaluation of clocks that resets $c \in r$ and keeps $c \in C \setminus r$ unchanged, and $u' \models Inv(l')$.

Uppaal [12] is a tool that supports modeling, simulation, and model checking of an extension of TA (*UTA*). In Uppaal, UTA are modeled as *templates* (see Fig. 2) that can be instantiated. UTA extends TA with data variables, synchronization channels, urgent and committed locations, etc. Variables and clocks can be set to certain values by the updates along the edges. In Uppaal, an update can be a comma-separated list of expressions, or a C-code style function that is implemented in the declaration of UTA. UTA locations can be marked as *urgent* (denoted by encircled "u") or *committed* (denoted by encircled "c"), indicating that time cannot progress in such locations. Committed locations are more restrictive, we explain it later. UTA can be composed in parallel as a *network* of UTA (NUTA) over a common set of clocks and actions [12]. In this paper, we model the communication between UTA via *synchronization channels* (e.g., a! and a? in Fig. 2) with rendezvous semantics: a sender (a!) synchronizes with a receiver (a?), provided that the sending and receiving edges are enabled, that is, their guards are satisfied. Committed locations require that the next edge to be traversed must start from a committed location in a NUTA. When multiple committed locations are ready to traverse, these transitions happen in a nondeterministic order, which means all possible orders need to be explored in an exhaustive model checking.

Figure 2 depicts an example of an NUTA in Uppaal, where blue circles are *locations* that are connected by directional *edges*. Double-circled locations are the *initial* locations (e.g., `L0`). `L3` is an urgent location, whereas `L4` is



(a) UTA template *TA1* (b) UTA template *TA2*

Fig. 2 An example of UTA templates in UPPAAL (Color figure online)

a committed location. On the edges, there are assignments resetting clocks (e.g., `c1:=0`) and updating data variables (e.g., `v1:=para1`), guards (e.g., `c1>=10`), and synchronization channels (e.g., `a!` and `a?`). At location `L1`, an invariant `c1<=15` regulates that clock `c1` must never exceed 15 time units. In UPPAAL, templates of UTA can have parameters (e.g., `para1` in TA1) that are assigned values when the UTA are instantiated. The UPPAAL model checker supports the verification of queries written in a decidable subset of (Timed) Computation Tree Logic ((T)CTL) [12]. The syntax of a (T)CTL formula consists of quantifiers over paths and path-specific temporal operators. There are two types of path quantifiers: the universal "A" meaning "for all paths", and the existential "E" denoting "there exists a path." We are interested in one path-specific temporal operator, that is, "Always" (`[]`) temporal operator. The UPPAAL queries that we verify in this paper are of the form: (i) **invariance**: `A[] p` means that for all paths, for all states in each path, `p` is satisfied, and (ii) **supremum evaluation**: `sup{con}:list` evaluates the supremum of the expressions in the `list` only when the condition (i.e., `con`) is *true*.

3 Feature selection and system abstraction

In this section, we present our modeling approach with the selected features and the conducted abstractions. We start this section by defining the ROS 2 system elements and their abstract representation for our formal models. While abstracting the systems, we focus on components and parameters needed to simulate and verify systems regarding callback latency, blocking, and input buffer overflow. Furthermore, we select features so that the models can be adapted in future work to verify more features.

ROS 2 systems consist of distributed nodes that communicate with each other through different communication channels. This system structure requires many system components and parameter settings, such as buffer sizes and quality of service for communication. When modeling a system for formal verification of selected properties, irrelevant components, and parameters can be removed, while other components can be simplified as abstract elements in the formal model.

Throughout this paper, we consider two approaches to modeling ROS 2 systems. The first models individual nodes while second approach involves modeling a system from a holistic perspective, including nodes, communication, and computation chains.

3.1 Verification goals

We have two verification goals, and thus select two features of ROS 2 systems, namely *input buffer sizes* and *callback latency*. The rest of the section introduces the verification goals and shows how to select features to model the system accordingly.

Callback latency Per definition, the latency of a callback is the maximum time from the release of a callback to the completion of the execution of the same callback instance. Hence, to simulate the callback latency, execution behavior needs to be modeled. Nevertheless, abstraction may be made to simplify the created models.

Buffer size and instance misses This paper proposes a way of verifying sufficient buffer sizes for ROS 2 callbacks. The input buffer in ROS 2 nodes stores incoming data instances, while the output buffer stores outgoing data. As a QoS parameter, buffer sizes are configurable for data-triggered callbacks that the user configures. A buffer overflow for any callback will lead to an instance miss, which can be critical. Hence, it is crucial to detect buffer overflows and select a sufficient buffer size. Especially timer callbacks are more likely to experience buffer overflow, due to their buffer size being one. In an older version of the executor (i.e., ExV1), timers are always prioritized, whereas in later versions (ExV2), timers can be blocked as low-priority callbacks, because they are only considered for execution when all callbacks in the previous ready set have been executed once. Therefore, the detection of buffer overflows, as in our model, is desirable.

3.2 ROS 2 components and abstractions

Given the stated verification goals, in the following, we present the ROS 2 components and their abstractions in the formal model.

The first ROS 2 component of a node influencing the execution is the *executor*. An executor is an ROS 2 internal scheduler and exists in every node. To simulate the execution of an ROS 2 system, we model the ROS 2 executor contained in each node. In this paper, we focus on the execution of main-line single-threaded executors, where two versions exist. We design the models such that simple switching between these two versions is possible by providing one model template for each version accordingly. The modularity of synchronized templates leaves future possibilities to extend the models

with templates for multithreaded executors, which is outside the scope of this paper.

As the execution of a specific callback depends on all other callbacks scheduled by the same executor, they also need to be included in the formal model. To determine the execution order of callbacks, an executor needs to know which callbacks have been released, and which are waiting to be released. Therefore, the callback models need to represent the release events and their time points. Other parameters that influence the execution, such as execution time and priorities of callbacks, need to be included as parameters of the UTA template of callbacks as well. Furthermore, the callback templates are designed to work with each executor model.

Since there are four types of callbacks in ROS 2, in our models, we want to find patterns and abstractions so that all callbacks can be modeled based on the same templates. Generally, each callback is released on a defined event. Such events are either the arrival of data or a signal from a system timer. Depending on the chosen approach, the communication channels, may be modeled using two different representations.

In our holistic approach, the arrival of data is modeled as a communication channel filling the input buffer, which triggers the execution of a data-triggered callback template. In the individual-node approach, the communication channels are abstracted away and modeled as arrays storing the releasing time points, which are used to fill the input buffers at the defined times. Accordingly, the callbacks are modeled either as sporadic or periodic. In both approaches, timers are modeled as sporadic or periodic callbacks with an input buffer size of one. The input buffer utilization is modeled in the callback templates, while the output buffer utilization is modeled in the communication channel template.

Allowing verification of buffer sizes, we need model components representing the input and output buffer. In the case of the selected verification, the actual data stores are optional, but the utilization and arrival times of the messages are not. Hence, the buffers can be abstracted as a counter together with an array of release times for simplicity. When data is received, the buffer's utilization is increased, and a callback is released. Depending on the policy, at the beginning of the execution of the appropriate callback, the oldest data point in the buffer is read and then removed, and the utilization counter is decreased. The variable showing the utilization allows running queries to check for specific utilization and overflow during all execution scenarios.

Besides the data-triggered callbacks, there are time-triggered callbacks. Their behavior is equivalent to a buffer size of one. In this way, time-triggered callbacks can use the same callback template as the data-triggered callbacks.

3.2.1 Modeling publisher–subscriber communication

Modeling publisher–subscriber communication involves at least one publishing, one subscribing node, and the communication channel itself. An example of a system containing two publishing and one subscribing node to one topic can be found in Fig. 3. In the first step, we explain how to model the communication between **Node0** and **Node1**, before showing the addition of more publishers and subscriptions. In the given scenario, **Node0** consists of one periodic timer, that publishes data to the topic `TopicH`. On the other hand, there is a subscription callback in **Node1**, subscribing to the topic. When modeling the sending node, first we need to instantiate an executor, using an executor template. Next, we can initialize the timer callback using the periodic callback template and link it to the executor with the according id. To model the topic, we use the Topic Template. Modeling **Node1**, we initialize a second executor with the Executor Template and the subscription callback with the Data Callback Template, linked to the executor. The sending of the data from the timer can be modeled by using synchronization between the topic and the callback template. The reception of the data is modeled using synchronization between the topic and the subscription callback.

Now, we are adding a new publishing node to the system (**Node0**). **Node0** consists of two periodic timers, each publishing to topic `TopicH`. First, the executor is modeled using the Executor Template. The timers are modeled using the periodic callback template and referenced to the executor. Even though the timers are publishing to the same topic as the timer in **Node0**, we need to create a new topic instance by using the Topic Template. This is due to the fact that the Topic Template contains the output buffer, which is individual to each node. Hence, the two timers in the same node can synchronize to the same topic instance by using the same `sender_id`. Two topic instances still can represent the same Topic, when they share the same receiver id. That means that if topic templates initiate the forwarding of data, the same callbacks are synchronized and will be triggered. When modeling multiple subscriptions to the same topic, each subscriber needs to refer to the subscription ID of the related topic and will be triggered by message transmission. In comparison to modeling multiple publishers with multiple topic instances, there is no need for a new topic instance for a new subscriber. This is due to the message transition to the subscription callback using a broadcast synchronization to all callbacks that use the same receiver.

3.2.2 Modeling of service–client communication

In this paragraph, we give a simple example of service-client communication. In ROS 2 systems, service–client communication uses a directed request–response communication

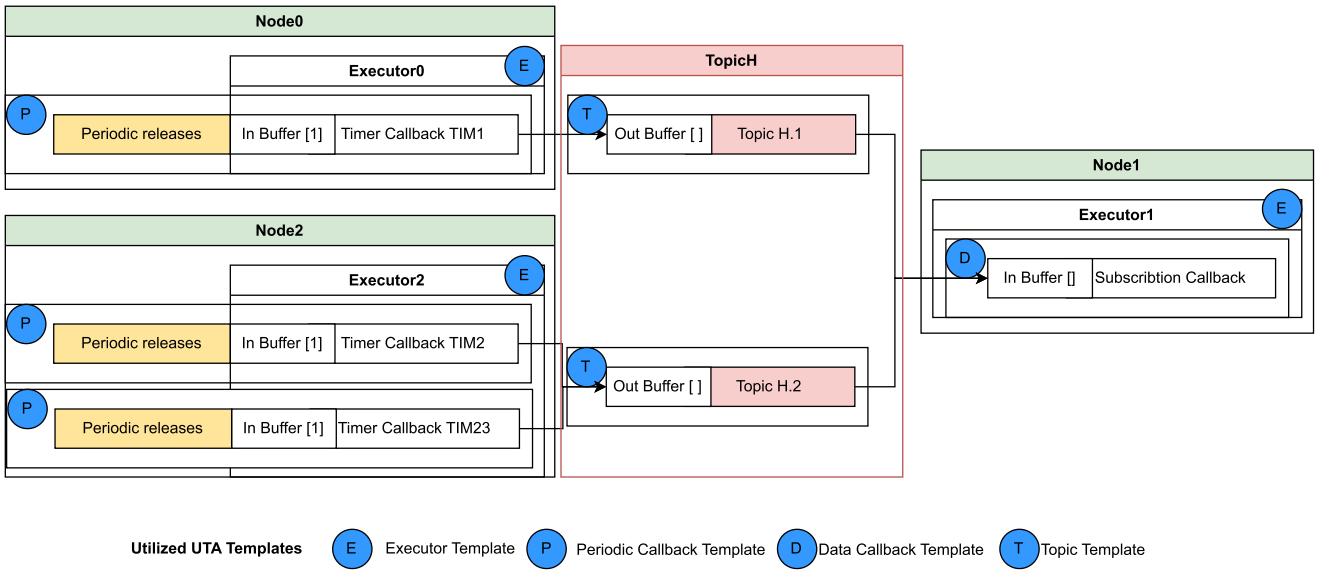


Fig. 3 Example of a publisher and subscriber node (Color figure online)

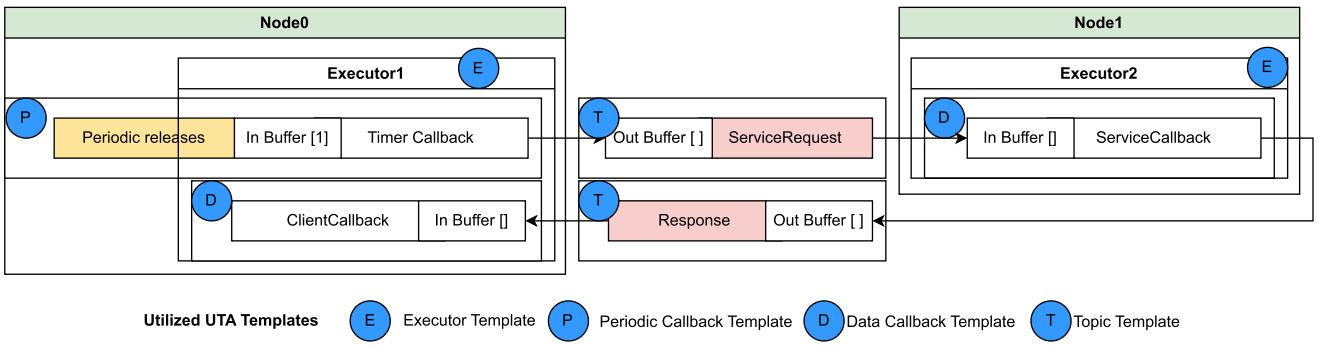


Fig. 4 Example of a model of a service and client communication between nodes (Color figure online)

pattern. An example can be found in Fig. 4. To trigger the execution of a service, a directed request is sent from a callback in the node that contains the client callback. In the given scenario, a periodic timer in **Node0** triggers a service request, where the service function is executed in **Node1**. Upon finishing the service, a response is sent back to **Node0**, triggering a client callback. To model the presented scenario, we can model **Node0** by initializing an executor using the executor template. Next, we create a periodic callback representing the timer by utilizing the periodic callback template, assigned to the executor. Furthermore, the client callback is created using the data callback template. Next, we model **Node1** by creating another executor and the service callback. To model the service client communication, we need two topic template instances. The first is used to receive the message from the timer callback and forward it to the service callback. The second topic instance is used to receive the message from the service callback and forward it to the client callback. Note, that with the current models, the modeling of one service to multiple clients is not possible due to the static sender and receiver synchronization in the topic UTA. Hence it is not possible to dynamically direct the messages and all connected clients would receive a response. Dynamic connection to topics is needed to answer the request to the correct callback. This is left to future work.

3.2.3 Modeling of a processing chain

After showing how to model communication, we give a simple example of a processing chain in this section. An example of a minimalistic processing chain can be found in Fig. 5. The following scenario is considered for modeling: **Node 0** consists of one periodic timer that executes every second and publishes data to a topic Topic1. In **Node1**, a subscription to Topic1 triggers a callback that executes and publishes a message in Topic2. Additionally, there is a periodic timer in **Node1**. In **Node0**, a subscription callback is triggered on messages in Topic2. The templates with the holistic and in-

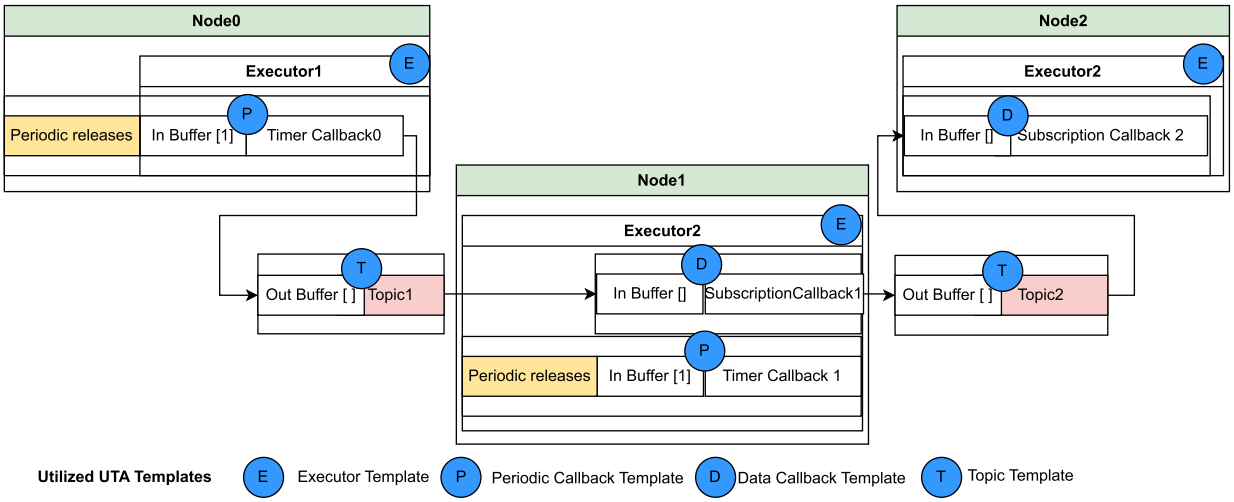


Fig. 5 Example of a processing chain (Color figure online)

dividual approaches can simulate the execution of callbacks in the Nodes. The approach of modeling the example is explained in the following paragraphs.

Holistic approach To model the scenario holistically, we can utilize the templates as shown with the blue circles in Fig. 5.

Individual approach Individual nodes can be extracted from the system and verified individually by utilizing the periodic and sporadic callback templates with the executor. Nevertheless, a more extensive system analysis is needed, as the release times of the callbacks need to be known as input parameters. While these parameters are simple to obtain for timers, message flow analysis needs to be conducted manually for data-triggered callbacks, such as subscriptions, to determine the release times of the corresponding callbacks in the node. As an example, the subscription callback in **Node0** can be modeled using a sporadic callback template instead of the data callback template.

Now, instead of modeling the communication, the release times of the subscription callback are taken as input parameters to the model template. These must be determined by analyzing the execution of **Node1**'s publishing to Topic1. As the timer in **Node 2** interferes with the publishing callback, even that must be considered when creating the release time array.

4 Modeling of ROS 2 component templates in UPPAAL

After explaining the abstractions that we introduce for our formal models, we present the created model templates in this section. As mentioned before, UPPAAL is used as a formal modeling and verification tool throughout this paper. A template-based modeling approach is chosen to allow a simple configuration of the models to verify ROS 2-based applications. As detailed in the previous sections, with the chosen abstractions and feature representation, there are two approaches to modeling ROS 2 applications. One approach is the holistic approach, where message passing is simulated, and the execution of nodes can be analyzed over processing chains. The other one is the individual-node approach, where communication is abstracted so that individual nodes can be verified simultaneously. In this section, we present and explain our proposed UTA templates needed for each approach, respectively. Generally, both approaches can be implemented using three different templates for callbacks, namely sporadic callback, periodic callback, and data-triggered callback.

4.1 Modeling of callbacks

In this work, we propose three different UTA templates to model callbacks, namely `Data Callback`, `Sporadic Callback`, `Periodic Callback`. While each template allows the modeling of all types of ROS 2 callbacks, they differ in the approach to modeling callback releases. A `Data Callback` template release is triggered by another UTA modeling the data communication. In contrast, `Sporadic Callback` and `Periodic Callback` releases are modeled with specified release times.

First of all, we create a framework with the needed input parameters for each callback. The declaration and parameters is shown in Listing 1. The following parameters are in common for the three templates. The parameter `id` is used to differentiate between callbacks of the same type, which is defined by the `type`, which is either `Subscription`,

`Service`, `Client`, or `Timer`. In addition, all callback templates contain the parameter `exec_time` to define the length of the execution and the `executorID` assigning a callback to an executor.

Listing 1 Formal declaration of callbacks

```
PeriodicCallback(id, exec_time, period, type,
    offset, BufferSize, publishers, publish_time[
    MAXPUB], publish_id[MAXPUB], executorID)

SporadicCallback(id, exec_time, length, releases[
    MAXX], type, BufferSize, publishers,
    publish_time[MAXPUB], publish_id[MAXPUB],
    executorID)

DataCallback(id, exec_time, topicID, type,
    BufferSize, publishers, publish_time[MAXPUB],
    publish_id[MAXPUB], executorID)
```

To publish data into the communication channels, each callback template contains the parameters `publishers`, stating to how many channels the callback is publishing. The parameters `publish_time[MAXPUB]`, and `publish_id[MAXPUB]` are arrays of a defined size stating at what time during execution the callback is publishing and to what communication channel.

As previously mentioned, the callbacks differ in their release. Hence, the parameters defining the release are differing. The periodic callback takes the `period` and an `offset` which delays the firs release as an input. The wall time callback takes the `length` defining how many releases are modeled and, `releases[MAXX]`, an array of defined size stating the absolute release times. The data callback takes the `topicID` to the triggering communication channel as an input.

Besides the modeled release mechanism, the internal logic of a callback model follows the same general structure. A simplified overview of the general structure of the callback templates is presented in Fig. 6. Note that the UTA presented in Fig. 6 is a generalization and only partially shows guards, updates, as well as invariants and parameters of the created UTA templates.[1]

Each callback has four different components, shown with blue boxes in Fig. 6. These are the `Waiting`, `Released`, `InReadySet` and `Execution` components. Each callback starts in the `Waiting` component, where the callback is not ready for scheduling, as it is waiting for a releasing event. When such an event occurs, the callback is `Released`, meaning it is ready to be polled into the ready set by the executor. Generally, an executor only considers callbacks included in such a ready set for scheduling. The polling is modeled with

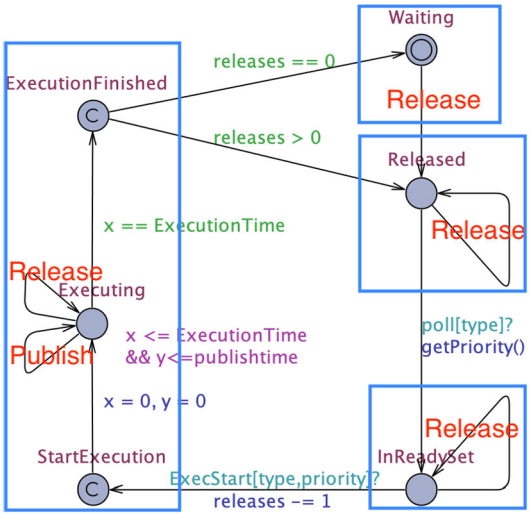


Fig. 6 UTA template overview representing callbacks (Color figure online)

a broadcast channel `poll`, synchronizing the callback with the executor, leading to the `InReadySet` location. Here, the callback is waiting to be scheduled for execution. A callback being scheduled by the executor is modeled by the edge from `InReadySet` to `StartExecution`, where the execution component of the UTA template starts.

The execution component contains three labeled locations for tracing and verification, i.e., `StartExecution` at the start of the execution, `Executing` during execution, and `ExecutionFinished` at the end of the execution. During an execution, a self-loop edge triggers data to be sent after a defined time. That must be possible for all callbacks using the holistic approach to trigger the communication channel templates to simulate data sending. On location `Executing` and its outgoing edge to `ExecutionFinished`, there is an invariant and a guard, respectively, forcing the callback model to stay at location `Executing` for the defined execution time. This models the duration of the callback execution. When an execution is finished, the UTA goes back to locations `Waiting` or `Released`, depending on the number of unhandled releases. As the callback type is essential for the execution order of callbacks, it is passed as an input parameter. To allow identification of the callbacks, each callback of the same type in the same executor needs to have a unique `id`. In an ROS 2 system, the priority of a callback is determined by the registration time, where the earliest registered callback has the highest priority. In our model, we use the `id` (Listing 1) parameter to set the priority. The callback with `id` 0 has the highest priority, representing the first initialized callback of each type. Furthermore, as a system can contain multiple nodes with executors, each callback must be assigned to an executor by referencing the corresponding `executorID`. The `executorID` is passed as a parameter to the template, but for simplicity reasons not shown in Fig. 6.

[1] The complete model is online: https://sites.google.com/view/pbvros2nodes.

One of the model's purposes is to verify buffer overflows, so the UTA contains a variable called *bufferUtil*. The variable represents the number of release instances contained in the buffer. Together with a parameter representing the buffer size, those parameters are sufficient to model the input buffer of a callback. In case of a new release, we verify if the release leads to a buffer overflow by checking if *bufferUtil* exceeds the buffer size. A designated boolean variable *Overflow* indicates if a buffer overflow has occurred. The variable is false by default and switched to true when an overflow occurs. When no overflow occurs, a new release instance is stored by increasing the variable *bufferUtil*. Given the implemented mechanics, *Overflow* can be used as a property to verify buffer overflow, while *bufferUtil* can be used to check the maximal buffer utilization.

Every location has a self-loop edge triggering the release mechanism, which might happen at each location of the callback. In Fig. 6, such release loops are marked with red `Release` statements. The release edges are guarded by the next release time for releases modeled by release times or the synchronized channel from the topic for releases modeled by communication in the holistic approach. When utilizing the `Release` edge, the described buffer mechanism is used to store the new release. Furthermore, a timer specific to the instance in the *ReleaseTimers* array is reset. The timer can later be used to determine how much time has elapsed since the release of the callback. This is used to verify the callback latency.

4.2 Modeling of communication channels

Communication in ROS 2 is organized over the so-called communication channels. These are either ROS 2 *topics* or *services*. The task of the channels is to transfer data from a sender to all receivers connected to the topic. In services, the request is directed where the client node sends a request, which will be executed in the server node and then sends a response that triggers a function in the client node.

In this paper, we assume the assignment of publishers and subscribers, as well as services and clients, to be static and conducted before runtime. Hence the mechanics of both types of communication can be generalized, and only one UTA template is needed to allow modeling of both kinds of communication.

Listing 2 shows the declaration of the topic template with the defined input parameters. The parameter `sender_id` allows synchronization to a sending callback using the same id to initialize data transfer. The parameter `receiver_id` allows the template to synchronize to a callback to simulate data reception. The parameters `delay` and `max_jitter` are used to simulate delays and communication jitters. Finally, the parameter `bufferSize` defines the output buffer size of the communication channel.

Listing 2 Formal Declaration of a communication channel

```
Topic(receiver_id, sender_id, delay, max_jitter,
    bufferSize)
```

The created UTA template for communication channels can be found in Fig. 7. Initially, the communication channel is in location *idle*, where it is waiting for communication to be initiated. The initiation of communication is modeled with a receiving synchronized channel, identified with a `sender_id`. Hence, the channel can be triggered from multiple edges in different nodes that use the same `sender_id`. The output buffer is modeled as a counter (`bufferUtil`) and a constant stating the size. The sending of data from a callback triggers a function (`bufferUpdate`) that updates the output buffer via a synchronized channel. In the function `bufferUpdate`, it is first checked if the buffer is full. If no overflow occurs, the value of the variable `bufferUtil` is increased by one. Next, when sending is initiated, two locations are used to simulate the communication delay and the maximum jitter. In this model, the communication delay is a constant, where the transmission of data is delayed for a fixed amount of time, modeled by the invariance and the guard on the leaving edge. The maximum jitter is a time interval describing a random delay to the transmission, modeled by the invariant in the location, but no guard on the leaving edge. When the delay and jitter are finished, the message is passed to the receiving callbacks. This is modeled by a broadcast channel initialized from the topic template, identified by the `receiver ID`. This allows multiple receivers to enable a synchronization with the same topic, by enabling the synchronized transition using the receiver ID. Additionally, the element is removed from the output buffer. After the sending, the UTA either moves to location Idle, when no data is in the buffer or restarts a transmission when data is left in the buffer.

4.3 Modeling of the executor

The executor is the last type of UTA template needed to model an ROS 2 system. The executor, as ROS 2's internal scheduling algorithm, determines in what order the released callbacks are executed. Generally, each node consists of at least one executor, and callbacks are assigned by the configuration by the developer. As shown in the previous sections, with ExV1 and ExV2, two versions of the single-threaded executor exist, dependent on the chosen ROS 2 distributions. The significant difference between the executor versions is the polling of timer callbacks, which is done continuously in ExV1 and only on defined polling points in ExV2. In this section, we present our approach to modeling the executor versions as UTA templates that can be instantiated with model parameters for an ROS 2 system simulation. In Listing 3 we show the definition of the two templates of the

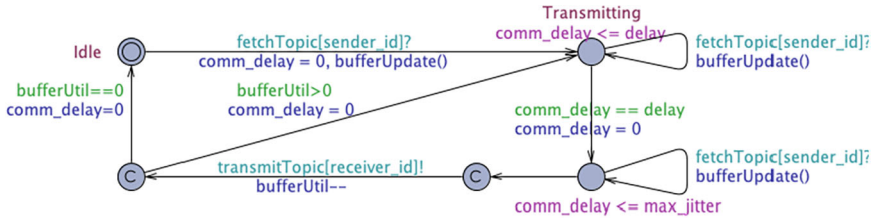


Fig. 7 UTA template overview representing communication channels (Color figure online)

executor with their input parameters. In our implementation, the executor requires two parameters. `Id` assigns a unique id to the executor to synchronize to the callback templates. Hence, multiple executors can be initialized in the system while still being able to identify and assign callbacks to the corresponding executor. Theoretically, it is possible to simulate executors of different types in the same system. The parameter `StopTime` is used to allow state space reduction while verifying a system. The parameter is used to cause a deadlock after the defined elapsed system time. We allow the exploration of the whole state space, by using any negative value.

Listing 3 Formal Declaration of an executor

```
ExecutorV1(id, StopTime)
ExecutorV2(id, StopTime)
```

Both versions of the executor algorithm have two distinctive phases. The first phase is the idle phase presented in Fig. 8 and is similar to both executor versions. The second phase, the execution phase, differs from ExV1 (Fig. 9(a)) to ExV2 (Fig. 9(b)).

Initially, the executor waits in its Idle location until at least one callback is released. The release of a callback utilizes a synchronized channel, identified with the executor ID (unique to each executor) to initiate the polling in the executor. In Fig. 8 it can be seen that two different edges with guards lead to two different polling locations. The difference is the simulation parameter `StopTime`. The parameter is implemented to allow verification in a given time interval (`StopTime` > 0) or infinite time (`StopTime` < 0). When selecting a defined interval, the model is forced into a deadlock by an invariance in the following Poll location when the defined stop time elapsed.

When reaching the idle location, the polling sequence is started. During the polling, synchronized channels are used to poll for released callbacks of any type. This sequence is conducted using committed locations so as not to interrupt the polling once it is started. Global variables track the number of released callbacks and their types. The counter for released callbacks of every type (*rs_tim*, *rs_sub*, *rs_srv*, *rs_cli*) is increased once the polling channels trigger a transition inside a callback. If no callback is released after the polling, and all the sets are zero, the executor will move to the waiting state again until at least one callback has been triggered. In case a callback has been released by the polling and at least one set is greater than zero, the executor model continues with the execution phase that is shown in Fig. 9(a) for ExV1 and Fig. 9(b) for ExV2.

During the execution phase of both versions of the executor, the execution order is determined, and the callbacks contained in the released sets are executed one at a time. In ROS 2 systems, Timer callbacks have the highest priority and are executed first. Subscribers, services, and clients follow them in descending order. Our model's execution phase starts with the location `FetchTimer`. If a timer is contained in the ready set *rs_tim*, the execution of the timer callback will be initialized. If multiple timers are available, that with the highest priority is executed first. The execution is initiated by the synchronized channel `ExecStart` leading to the `ExecuteTimer` location. The location will be held until the execution time of the callback elapses and the synchronized transition `ExecDone` is triggered by the callback. Then, the ready set is decreased as the execution is finished. The procedure is followed for all timer callbacks in the ready set before continuing with the subscribers, services, and clients. The difference between ExV1 and ExV2 is that in ExV1, once the execution of any callback has been finished, a polling sequence is conducted for timers only. If any timer has been released during that polling instance, the executor will go back to the `FetchTimer` location and execute the timer before continuing with callbacks of other types.

5 Application of pattern-based verification

After creating the UTA templates, this section explains how they can be used to model an ROS 2 application and how to apply UPPAAL queries to verify callback latency and buffer sizes.

5.1 System composition

When modeling a system, the templates presented in Sect. 4 are used to compose the formal model.

Templates are instantiated in the systems definition and filled with the required parameters. Listing 4 shows the instantiation of the example of a publisher–subscription communication given in Sect. 3.2.1 using the holistic approach.

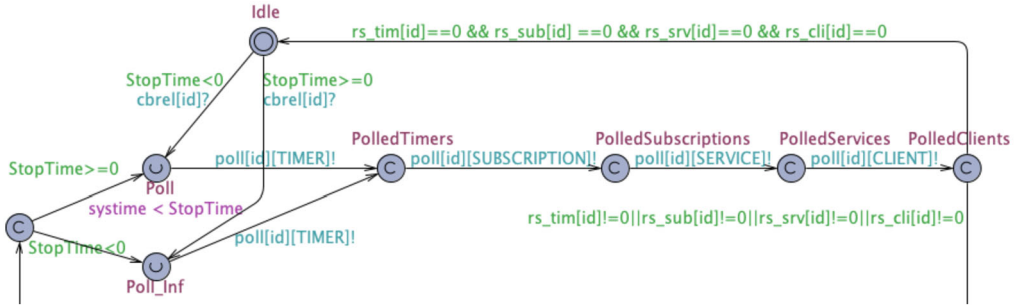
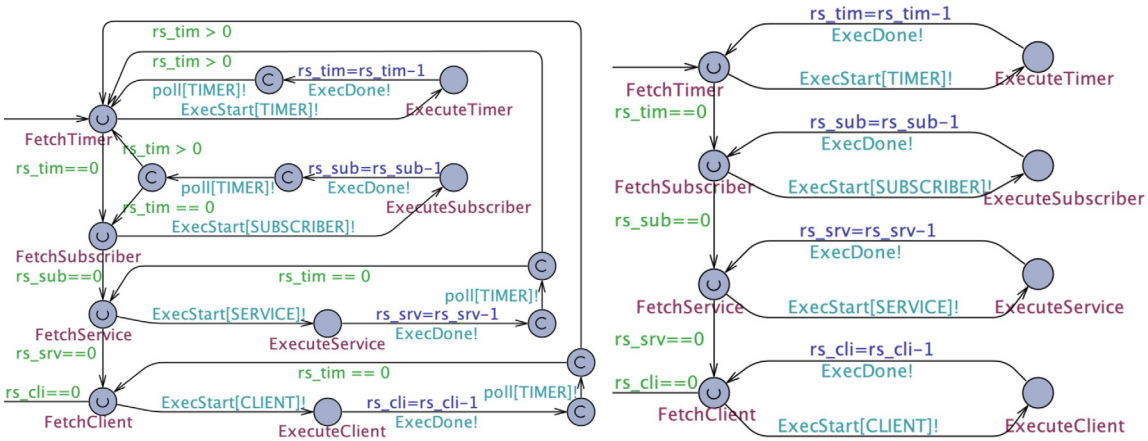


Fig. 8 UTA template extract showing the polling of callbacks in the executor (Color figure online)



(a) UTA template extract showing the execution component of ExV1

(b) UTA template extract showing the execution component of ExV2

Fig. 9 Execution component of the executor UTA templates (Color figure online)

Listing 4 Example of an UPPAAL system declaration for a publisher–subscriber communication

```
const int StopTime = -1;
TopicH = Topic(0,0,0,0,10);
Node0 = ExecutorV2(0, StopTime);
const int sequence1[MAXPUB]={0,0,0,0,0,0,0,0,0,0};
Timer = PeriodicCallback(0, 5, 10, TIMER, 13, 1,
    1, sequence1, sequence1, 0);
Node1 = ExecutorV1(1, StopTime);
H = DataCallback(0, 5, 0, SUBSCRIBER, 10, 0,
    emptyArrayPub, emptyArrayPub, 1);
system Node0, Node1, H, Timer, TopicH;
```

First, the simulation time is defined as a parameter describing for what time the system should be verified. The value −1 stands for whole-state space exploration. Next, the topic is initialized using the parameters shown before. Next, the node executors are initialized with individual IDs and the simulation time. Next, one periodic callback is initialized for the timer in **Node0**, and a data callback for the subscription callback in **Node1**. Note that sequence1 stands for the time the callback publishes its message during execution. Finally, the system is composed of the created components.

Further examples for the service–client communication in Sect. 3.2.2 and the processing chain in Sect. 3.2.3 using the holistic and individual approach can be found in the appendix (Listings 5–7).

5.1.1 Simulation and verification

In this section, we show how to use the model to simulate and verify the execution of ROS 2-based applications. In the first step, the UPPAAL simulator can either manually select enables transitions to generate possible system traces or randomly iterate through possible execution. Besides the current locations of the used automata, system variables, and their values can be seen at any time. By utilizing the inbuilt UPPAAL verifier, exhaustive model checking on the UTA model can be performed. In this paper, we focus on verifying the input and output buffer overflow, such as the callback latency. Each callback contains a variable `Overflow`, initially false and set to true when an overflow occurs. Therefore, the absence of a buffer overflow can be verified by checking that for all states in every possible path (`A[]`), the variable `Overflow` in a selected callback `cb` or topic `tp` is `false`.

Examples of such queries can be found in Eq. (2), where XX either has to be replaced with the callback or the topic. A buffer overflow is always equivalent to an instance missed by a callback; hence, this query verifies missing callback instances.

```
A[] XX.Overflow==false (2)
```

When the stated invariance does not hold, a buffer overflow is detected and a counterexample with a simulation trace can be generated, ending with the invariance not being valid. The trace with the internal variables can then be analyzed. Depending on the UPPAAL setting, either the fastest, the longest, or a random trace can be generated. When the property of an absence of a buffer overflow is verified, a second query can be used to determine the maximum utilization of the buffer. This can be done by simply checking the supremum of the variable `bufferUtil` for a callback `cb` or a topic `tp`, as shown in Eq. (3). XX needs to replaced with either the callback or the topic. This value can be used to determine parameters for buffer size sufficient to hold all incoming and outgoing releases for the given system parameters.

```
sup:XX.bufferUtil (3)
```

As the latency is defined as the maximum time from the release of an instance to the end of the execution of the same instance, we need to know the release time and the end of the execution of any callback instance. Therefore, we included an array of timers and counters that keep track of the release times of each callback instance The end of the execution is marked by a location called `ExecutionFinished`. Using a location-specific supremum evaluation for a callback `cb` at the location `ExecutionFinished` for a timer reset at the release of the callback instance, the longest callback latency can be determined as follows:

```
sup{cb.ExecutionFinished}:cb.relTim[cb.relcnt-1] (4)
```

To generate an example trace containing the maximum latency for the selected callback, Eq. (5) can be used with `XX` being the determined latency. The query checks if, in any of the possible paths in any state, the execution of the callback is finished and the latency clock has the specific value. If satisfied, the trace can be generated automatically.

```
E<> cb.ExecutionFinished and cb.relTim[cb.relcnt-1]==XX (5)
```

6 Evaluation

In this section, we evaluate the model templates and their application to simulate and verify ROS 2-based applications. Therefore, we compare the verification results with a real ROS 2 execution on two scenarios of message passing between two nodes in two scenarios. The first scenario (SC1) focuses on sporadic callbacks and message passing. With the second scenario (SC2), we demonstrate buffer overflow using a combination of sporadic and periodic execution. In both these scenarios we use two different approaches, namely *Individual* and *Holistic*, as described under each scenario. The ROS 2 execution traces of actual executions are created by running the test system on a single computer using Docker to create a controlled environment. We conduct the experiments in ROS 2 *Dashing* (the last distribution that contains ExV1), *Eloquent* (the first distribution that contains ExV2), and *Humble* (the latest long-time stable distribution that contains ExV2). A trace is created with each callback, recording the start and end of execution in the ROS 2 log.

After evaluating the correctness of our models with real ROS 2 examples, with a third scenario (SC3), we give a simple example of a message passing between two nodes that is used to demonstrate the verification of output buffer overflow and the effects of delays and message jitters. As this is hard to show in a real system with current analysis tools, we only show the application using our models.

6.1 Scenario 1 (SC1): sporadic callbacks

In this scenario, one node utilizes sporadic timers to send message sequences over different communication channels to a second node, the execution of which is of interest in this evaluation. The scenario is taken from a previous evaluation of the scheduling algorithm by Casini et al. [8]. Communication channels are named after the priorities of the receiving callback in Node1. Therefore, topics are called *H, M, L* (High, Medium, Low) and services *SH, SM, and SL* (Service High, Service Medium, Service Low). To make the execution times of the callbacks deterministic and to simplify evaluation, we implement a function that runs for 500 ms. This function is called from each callback in `Node1`. Additional time overhead through variable instantiation and the function call within the callback is negligible compared to the 500 ms execution time of the function. In addition to the data callbacks in Node1, there are four timer callbacks. Except for the timer callbacks with a static buffer size of one, all input and output buffer sizes are set to 10. Two message sequences (*S0* and *S1*) are transmitted from Node0 to Node1: *S0* is released after 0 seconds and consists of data being sent in the following communication channels: *<L; M; H; SH; SL; L; M; H; SH; SL>*; *S1* is released after 1.5 seconds and triggers data sending in the channels $< SM; SM; H >$. Additionally to the message sequences, the timers T0 and T1 release after 0.2 seconds, while T2 and T3 release after 2.3 seconds. A trace of an observed execution of the scenario with ExV1 and ExV2 in a ROS 2 system can be found in Fig. 10. In ExV1 (upper trace), timers are scheduled immediately after their

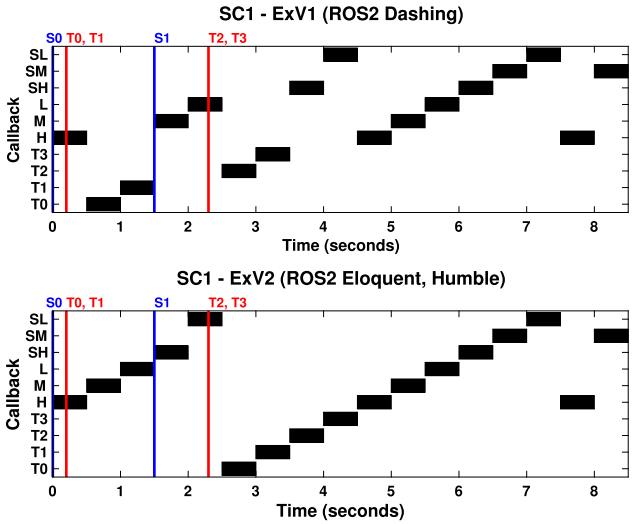


Fig. 10 Execution diagram of Node1 executing SC1 on different ROS 2 distributions. Sequence releases are marked in blue and releases of timers in red (Color figure online)

release, and the execution of the current callback is finished. In ExV2 (bottom trace), timers are scheduled after executing all remaining callbacks in the ready set.

6.1.1 Modeling and verification of the system

In this section, we present how the system in this scenario is modeled via two types of approaches, i.e., the individual approach and the holistic approach. Then, we present how model checking is employed to verify the models as well as the verification result.

Individual Approach. Using the individual approach for modeling the execution of Node1, communication is abstracted to release times, which can be obtained by analyzing the communication. For example, it can be determined that callback H is released twice at time 0 and once at 1.5 seconds. After manually obtaining the release times, all callbacks in Node1 are modeled using the sporadic callback templates. Furthermore, all callbacks are linked to the same executor model, as they are executed in the same node. We set the simulation time parameter to −1 to simulate the scenario until the end.

Holistic Approach. In the holistic approach, we include the communication between the Nodes. As we are interested in the execution of Node1, we can still make some abstractions (such as removing the client callback in Node0). However, we set up Node0 with an executor and two timer callbacks. The timer callbacks release messages through different communication channels. Therefore, we must use the topic templates to create one model for each communication channel. We model the timers to release the callbacks in Node0 with an execution time of 0 and the message send time after 0 seconds. This abstraction is to model the scenario accurately, assuming that messages have arrived at time 0. Now, we model Node1 with an individual executor and the data-triggered callback templates, with one instance for each data-triggered callback linked to the communication channel. Furthermore, we instantiate the timers using the sporadic callback templates.

Table 1 Determined Latencies using Eq. (4) SC1 for the individual approach (UI) and the holistic approach (UH)

	ExV1			ExV2		
Callback	Real	UI	UH	Real	UI	UH
T0	0.8	0.8	0.8	2.8	2.8	2.8
T1	1.3	1.3	1.3	3.3	3.3	3.3
T2	0.7	0.7	0.7	1.7	1.7	1.7
T3	1.2	1.2	1.2	2.2	2.2	2.2
H	6.5	6.5	6.5	6.5	6.5	6.5
M	5.5	5.5	5.5	5.5	5.5	5.5
L	6.0	6.0	6.0	6.0	6.0	6.0
SH	6.5	6.5	6.5	6.5	6.5	6.5
SM	7.0	7.0	7.0	7.0	7.0	7.0
SL	7.5	7.5	7.5	7.5	7.5	7.5

Verification of Latency and Buffer Overflow. We model the system with all messages being received and, according to callbacks, being released at time 0 before the executor starts to schedule the callbacks. We model this by giving the executor the lowest priority in the system. Now, we employ UPPAAL to perform model checking against the queries in the form of Query (2) for checking the possible eventual buffer overflow. Further, we calculate the callback latencies by running queries in the form of Query (4). In this scenario, the buffer sizes are set to 10, so that there should be no buffer overflow. We use the created model to verify the property for each communication channel's input and output buffers.

Table 1 shows the observed results for the maximum latency. It can be seen that both approaches show the same maximum latencies as observed in the real system execution.

Naturally, the checking of the Holistic Approach takes longer (up to 7.3 seconds for the holistic approach compared to 0.5 seconds for one query in the individual approach) due to the system complexity.

6.1.2 System simulation and trace analysis

Next, we use the symbolic simulator in UPPAAL to generate random traces of the model. We verify the correctness of the models by comparing the simulated execution traces with the observed ones. We can also confirm that the traces match the execution.

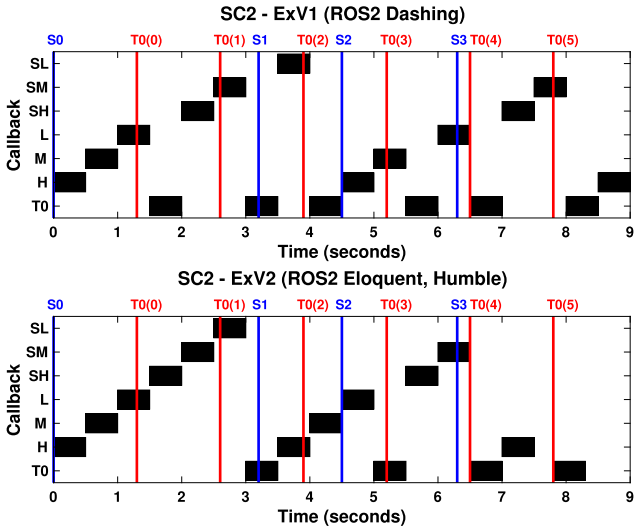


Fig. 11 Execution diagram of Node1 executing SC2 on different ROS 2 distributions. Sequence releases are marked in blue and releases of timers in red (Color figure online)

6.2 Scenario 2 (SC2): periodic callbacks and input buffer overflow

In SC2, we create a system similar to SC1 with just one change: Node1 contains only one instead of four timers together with the service and subscription callbacks. The timer is set to be released periodically every 1.3 seconds. Four message sequences (*S0, S1, S2, S3*) are sent from Node1 to Node 2: *S0* is released after 0 seconds, triggering $< H; M; L; SH; SM; SL >$; *S1* is released after 3.2 seconds and triggers $< H; M; L >$; *S2* is released after 4.5 seconds and triggers $< SH; SM >$; *S3* is released after 6.3 seconds and triggers $< H >$. A resulting execution trace in a real ROS 2 system with ExV1 and ExV2 can be found in Fig. 11. In ExV2, it can be seen that timer `T0` only executes four times, even though it is released six times. This is caused by an input buffer overflow as a result of a block of the callback that spans more than one period. In ExV1, `T0` is executed for each release when a buffer overflow occurs.

6.2.1 Modeling of the system

We model the given scenario utilizing the created UTA templates. We create models for both the individual and the holistic approach.

During a first verification round, we assume that all messages are being received and the corresponding callbacks are released at 0 seconds before the executor schedules the callbacks (`A1`). We model this by giving the executor the lowest priority in the system.

In a second verification round, we assume that messages must not be received before the scheduling of the executor at 0 seconds (`A1`). Hence, all system components have the same priority.

Table 2 Determined Latencies using Query (4) in SC2 with the individual approach (UI), the holistic approach (UH) for the two assumptions (A1,A2)

	ExV1					ExV2				
		A1		A2			A1		A2	
Callback	Real	UI	UH	UI	UH	Real	UI	UH	UI	UH
T0	0.9	1.0	1.0	1.0	1.0	2.2	2.2	2.2	2.2	2.2
H	2.7	2.7	2.7	4.0	4.0	1.2	1.2	1.2	3.5	3.5
M	2.3	2.3	2.3	5.0	5.0	1.3	1.3	1.3	3.5	3.5
L	3.3	3.3	3.3	5.5	5.5	1.8	1.8	1.8	3.5	3.5
SH	3.0	3.0	3.5	6.5	6.5	2.0	2.0	2.0	3.5	3.5
SM	3.5	3.5	4.5	6.5	6.5	2.5	2.5	2.5	3.5	3.5
SL	4.0	4.0	4.0	6.5	6.5	3.0	3.0	3.0	3.5	3.5

Individual Approach. To model the behavior of callbacks in Node1, we use sporadic callbacks for all callbacks triggered by messages. We initialize the release times to when the data will arrive in the node. The timer is modeled as a periodic timer released every 1.3 seconds, with an offset of 1.3 seconds, as it is not released at time 0. We link all callbacks to one instance of an executor.

Holistic Approach. Following the holistic approach, we implement the first node by creating four timers that send the sequences once they are executed. The timer callbacks release the messages according to the scenario through the different communication channels. Therefore, we use the topic templates to create one model for each communication channel. Besides using one periodic timer with a period of 1.3 seconds in Node1, the configuration is similar to SC1.

Verification of Latency and Buffer Overflow. In the first step, we use UPPAAL to check for buffer overflow in all callbacks in both approaches for both scenarios. Using the UPPAAL queries in the form of Query (2), we detect a buffer overflow in ExV2 with both approaches and assumptions. That verification result corresponds to the actual execution. Now, we use UPPAAL to calculate the maximum latency by checking queries in the form of Query (4). The results are in Table 2. Naturally, the checking of the Holistic Approach takes longer (up to 1.5 seconds for A1 and 66 seconds for A2 using the holistic approach compared to 0.2 seconds for A1 and 1.5 seconds for A2 using the individual approach) due to the system complexity. The results show that in ExV1, with A1, both approaches detect a longer maximum latency for T0 than the observed execution trace (red). Furthermore, the holistic approach determines even higher latency for SH and SM (red). For ExV2, no deviation can be seen with A1. Using A2, both approaches (holistic and individual) detect higher latencies for almost all callbacks (blue). The deviations and their causes are analyzed further in the following section.

6.2.2 System simulation and trace analysis

To analyze the determined differences, we use the verifier to generate system traces that we can compare and analyze

Table 3 Traces for the different scenarios using Query (5) (Color figure online)

```
Observed Execution ExV1:
H;M;L;T;SH;SM;T;SL;T;H;M;T;L;T;SH;SM;T;H
Generated Trace for ExV1, A1, T0 Latency=1 UI and UH:
H;M;L;T;SH;SM;T;SL;T;H;M;T;L;SH;T;...
Generated Trace for ExV1, A1, SH Latency=3.5 UH:
H;M;L;T;SH;SM;T;SL;T;H;M;T;L;T;H;SH;...
Generated Trace for ExV1, A1, SM Latency=4.5 UH:
H;M;L;T;SH;SM;T;SL;T;H;M;T;L;T;H;SH;T;SM
Generated Trace for ExV1, A2, SL Latency=6.5 UI and UH:
H;M;L;T;SH;SM;T;H;T;M;L;T;SL;...
```

using Query (5). Generally, the observed execution shows only one possible execution path, as it is only one execution. Other paths may show up in a real scenario after repeated execution. Nevertheless, if the models are exhaustive, all possible traces of an observed execution should be able to be observed in the possible traces generated by the model. Therefore, in the first step, we validate the faithfulness of our formal models by checking if the observed trace from the observed execution can be generated using the models, which we can confirm.

Next, we use the UPPAAL verifier to generate system traces of the cases where the higher latencies are determined as the observed execution. In the following Table 3, we present the execution order from an observed execution compared with the execution order from sample traces for the higher latencies determined by the models.

The explanation for the difference in the observed latency and the maximum latency determined by the individual and the holistic approach is as follows: The order of execution of the last instance of callback `SH` and `T0` can vary. That variation is caused by the release of a callback and the simultaneous polling of an executor. The execution order is nondeterministic and depends on which path the model takes first, meaning that the execution order might change. The experiment in the observed execution only demonstrates one of the possible paths. The model-checking is exhaustive, so the model detects additional paths that are possible based on the configuration. This highlights the usefulness of the model checking. Nevertheless, the same settings in the models should be able to generate the same results. The differences between the Individual and Holistic approaches in A1 are based on the following assumption: at time 0, the corresponding messages should be received before the executor schedules, and we set the executor to the lowest priority. This means that when messages are released at a specific time point, the callback is released before the executor performs any actions like polling or scheduling. The individual approach always takes precedence when multiple events occur simultaneously: messages are being transferred first, and callbacks are released before the executor executes. However, this is only valid for the initialization in the holistic approach. Hence, in a later phase, the message passing is simulated through a callback sending and a callback receiving, which are actions dependent on different executors. Depending on which executor is modeled to execute first, the assumption that messages are being sent and received first is no longer valid after the initial stage of the verification. Consequently, this overapproximation can identify even more potential system faults.

Table 4 Results using Query (2), and Query (4) in SC3

BufferSize	3			2		
Delay	0	1	0	0	1	0
Max Jitter	0	0	1	0	0	1
Possible Overflow	No	No	No	Yes	Yes	Yes
Max Latency	15	13	15	15	9	15

With assumption 2, we observe that even the individual approach, without prioritization of the executors, can generate the same approximation of the system execution paths as the holistic approach. By analyzing the obtained results and traces further, we conclude that the generated paths for the maximum latencies are theoretically possible, and therefore, the determined latencies are achievable. Furthermore, the errors discovered by model checking are potential errors (which might appear sometime) in the system, as UPPAAL checks all possible model executions that might not manifest in the observed ROS 2 traces. However, they could manifest (hence, be observed) under the right conditions.

6.3 Scenario 3 (SC3): output buffer overflow

With SC3, we show the application of modeling and verification communication delay and latency and an output buffer overflow. As with the current interfacing applications of ROS 2, it is not possible to create a controlled environment to make this repeatedly observable in an actual execution; we only focus on a theoretical scenario. In our UTA templates, the output buffer is included in the topic template. Hence, we can only run verification using the holistic approach that includes the topic template and a data triggered callback UTA. In this scenario, we implement one callback that is released once and instantly sends three data points to a topic. In the first iteration, we set the topic's output buffer size to 3 before setting it to 2 in the second iteration. For each iteration, we first simulate with zero jitter and zero delay, then with a delay of 1 and no jitter, and then with a jitter of 1 and no delay. The verification outcomes can be found in Table 4. The difference between the delay and the jitter is that the delay applies to all instances of communication, while the jitter is between zero and the selected number. The important finding is that with a buffer size of 2 and a delay of 1, the maximum latency is actually reduced compared to the buffer size of 3 with a delay of 1. That might lead to the conclusion

that a buffer size of 2 might actually be better. Nevertheless, in the given case, the lower latency can only be observed when a buffer overflow occurs. The overflow leads to one less execution of the callback. If the callback would have been executed, a higher latency would have been observed. Therefore, it is important to check both, buffer overflow and the latency when adapting the buffer size.

7 Answers to the research questions

In this section we address the research questions based on our implementation and experimental evaluation.

RQ1: Given the two versions ExV1 and ExV2 of single-threaded executors in ROS 2, how can we ensure the correctness of a design of ROS 2-based systems concerning the behavior of timer callbacks, callback latency, and the sizes of input buffers of callbacks?

Answer: In this paper, we apply pattern-based verification to verify the correctness of ROS 2 applications. We create formal model templates of ROS 2 components that can be composed of system models. Then, model checking can be performed on the created models to verify callback latency and buffer sizes.

RQ2: What are the patterns for modeling ROS 2 systems that can be reused in verification?

Answer: Our created model templates allow two verification approaches with different levels of abstraction. In the first, nodes are modeled individually. The execution of other nodes and communication is replaced by release time parameters that represent the arrival time of data. In the given approach, every callback can be modeled as a sporadic callback with defined release times or periodic callbacks with defined periods. While timer callbacks have a static buffer size of one, the buffer size for the other types of callbacks is configurable. In the second approach, systems are modeled holistically. Callbacks and their communication channels are modeled as system components. Nevertheless, only the events of message passing are of interest, not the transmitted data. In this approach, even the modeling of output buffers is possible as abstract message transport is simulated. In both approaches, other execution-relevant parameters, such as buffer sizes, execution times, and types of ROS 2 callbacks, are passed as arguments to the created templates. The actual data communicated over the network and the execution logic of the callbacks are not relevant to the identified verification goal. Hence, buffers can be modeled as counters. Different templates can be used to model the semantics of the ROS 2 mainline executor versions, all of which work with the created callback templates. We validate the determined patterns by comparing the simulation traces of the model to the actual system execution.

RQ3: Can model checking find the errors of ROS 2 systems which are shown in experiments, but also reveal potential errors that are not discovered via experimental evaluation?

Answer: In our experimental evaluation, we compared verification results to observed system execution. First, we ensured that all observed traces in reality were contained in the set of traces from the models. Hence, we could verify that the models could find the presented errors in the scenario. Furthermore, with both created modeling approaches, it was possible to determine traces that lead to higher latency than the initially observed one. We manually analyzed those traces to confirm that these traces are theoretically possible in a real system. As they lead to a higher latency, they might be seen as potential system errors. Therefore, we can confirm that model checking can find potential errors in an ROS 2 application that might not be discovered in experiments.

RQ4: How does the approach of modeling an application holistically differ from the approach of modeling callbacks individually?

Answer: Modeling a system with the individual node approach requires extensive analysis and understanding of the system. Especially in processing chains, the execution of previous nodes needs to be understood to determine the release times of the callbacks in the node under evaluation. Furthermore, effects such as communication jitters are more complicated to model, and output buffers are excluded. Nevertheless, executing the verification computation takes less time and resources, as the state space is minimal. When modeling a system with a holistic approach, more effort is required. More components must be included in the model, increasing the state space. Hence, the computation time increases drastically for complex systems. Both methods obtained the same critical paths and latencies in the experiments without the system execution assumptions. With assumptions, the holistic approach detected more critical paths.

8 Related work

The ROS 2 scheduling problems that we are studying in this paper have been identified within the real-time systems community, as follows. Blass et al. [6] assume the executor model that includes timers as part of the ready set, and carry out the response time analysis for callbacks in a node, using the single-threaded executor and reservation-based scheduling. Investigating the same problem, Tang et al. [18] use the executor model with timers excluded from the ready set instead. Both works are pivotal for the research presented in this paper, and the experiments conducted illustrate the differences in the revised executor model, consistently.

Even though callback latency verification resembles WCET analysis, the former in ROS 2 extends WCET analysis

to distributed systems with complex internode communication and diverse scheduling behaviors. While WCET typically assumes predictable task preemption and execution, ROS 2 introduces unique challenges due to the following: (i) callback execution often depends on asynchronous message passing and scheduling policies, making direct reuse of WCET techniques challenging, (ii) ROS 2's executor introduces additional complexity. For example, different versions of ROS 2 executors (e.g., ExV1, ExV2) exhibit distinct polling and scheduling mechanisms that directly impact callback latencies, and (iii) unlike traditional WCET analysis, ROS 2 systems must consider buffer sizes and potential overflows, particularly for time-triggered callbacks, as highlighted in our work.

The existing related work that uses UPPAAL for WCET analysis [1, 9] reduces to computing the longest path (timewise) in a network of timed automata and does not account for ROS 2-specific features such as dynamic executors, topic-based communication, or data-driven callbacks. In comparison, our work addresses the mentioned challenges, using modular TA templates for modeling ROS 2 nodes, their interactions, and timing behaviors. These templates facilitate the analysis of callback latencies and buffer overflows, as described in Sects. 3 and 4 of our paper.

The formal verification of both the communication and computational aspects of ROS 2 has garnered considerable attention lately. Halder et al. [11] present a methodology aiming to model and validate ROS 2 communication among nodes using UPPAAL. In their approach, akin to ours, they incorporate low-level parameters like queue sizes and timeouts into their TA models to verify queue overflow. However, their methodology lacks the consideration of callback latency verification and does not encompass the two models of ROS 2 single-threaded executors. Furthermore, their work does not include the validation of formal models, a step which we undertake in our research through the utilization of simulation experiment results.

Carvalho et al. [7] introduce a model-checking technique aimed at verifying system-wide safety properties in message-passing architectures. This approach is rooted in formalizing ROS launch configurations and the loosely defined behaviors of individual nodes, utilizing an Alloy extension named Electrum and its Analyzer. Electrum models are automatically generated from configurations obtained via continuous integration and specifications provided by domain experts. Their methodology emphasizes high-level architectural verification of message passing. In contrast, our research delves into lower-level model checking, specifically examining the scheduling of single-threaded executors under two distinct timer semantics observed in various ROS 2 distributions. Webster et al. [20] propose a formal verification methodology for industrial robotic programs utilizing the SPIN model checker. Their focus lies in behavioral refinement and verifying selected robot requirements. Although their solution is applied to an existing personal robotic system, it is not ROS-specific and primarily targets the verification of high-level decision-making rules.

Anand and Knepper [4] present ROSCoq, a Coq framework tailored for the development of certified systems in ROS. This framework leverages CoRN's theory of constructive real analysis to address computations involving real numbers. Their methodology follows a "correct-by-construction" approach, differing from ours but offering complementary perspectives. However, our focus lies in verifying diverse timer scheduling mechanisms present in ROS 2 distributions, an aspect not covered in their work.

9 Conclusion and future work

In this paper, we utilize pattern-based verification to ensure the correctness of ROS 2-based applications using UPPAAL. In our approach, formal specification serves as the foundation for analyzing ROS 2 systems, ensuring precision and eliminating ambiguity. Our work extends formal specification by creating reusable TA templates for various components of ROS 2 systems, including: (i) callback execution, (ii) executor behavior (single-threaded, ExV1 and ExV2 versions), and (iii) communication mechanisms (topics and services). These templates enable modular and systematic modeling, reducing the complexity and effort required to model new ROS 2 systems. Model checking is employed in our approach to exhaustively explore the state space of an ROS 2 application model to ensure compliance with specified properties, such as: a) Callback Latency: Model checking ensures that callback latencies remain within acceptable limits by verifying the time elapsed between callback release and completion; b) Buffer Overflows: The tool checks for scenarios where input or output buffer overflows might occur, potentially leading to missed callback instances or loss of data; and c) Deadlocks and Scheduling Issues: Model checking identifies issues like deadlocks in callback execution or scheduling anomalies caused by nonpreemptive executors.

Therefore, we create UTA templates that can be initialized with parameters composed of system models. The system models can be used to generate system traces and to execute verification queries to ensure the correctness of system design regarding buffer sizes and callback latency. The created templates allow two approaches to verification. In the first, nodes are modeled individually, while in the second approach, the system is modeled holistically. While the first approach requires more initial effort in system analysis, it is faster in executing verification queries. The second approach requires more effort in modeling actions but less knowledge of ROS 2 and the system. Two versions of the mainline

single-threaded executor are included in the models. In our experimental evaluation, we show verification's usefulness and that verification can find potential system errors that might be overlooked in experiments. System traces leading to such system errors can be created automatically and used to analyze the issues. The created models only include the ROS 2 internal execution, assuming 100% availability of the executing thread. The detected system errors remain possible when the computation reservation is decreased. Nevertheless, new system errors might exist that are not covered by our models. The models allow future improvements and refinements to verify other properties, such as data age. In our current work, we are leveraging UPPAAL, which supports a subset of CTL. Within the given subset of CTL, an extension towards liveness properties would be desirable and is left to future work. It is also very important to conduct more evaluation on scalability of the approach. When scaling to many nodes in a network, the verification might get complex fast. Hence, state space reduction and optimization are desired. Also, other approaches using different formal verification tools and formal modeling languages are of interest for extending the work.

Furthermore, ROS 2 also supports multithreaded executors, which can be implemented as alternative executor templates in future versions. Additionally, the created approach to modeling communication channels can be improved by modeling dynamic connections under runtime for a better model of service–client communication. More extensive analysis on formal semantics of ROS 2, the level of overapproximation of the created models and formal proof of bisimulation is desirable for additional future work. Overall, the verification and modeling approach proposed in this paper is a first step toward pattern-based verification of ROS 2-based applications and distributed robotic systems.

Appendix: System model declarations in UPPAAL

Listing 5 Example of a UPPAAL system declaration for a service client communication

```
const int StopTime = -1;
Request = Topic(0,0,0,0,10);
Response = Topic(1,1,0,0,10);
Node0 = ExecutorV2(0, StopTime);
const int sequence1[MAXPUB]={0,0,0,0,0,0,0,0,0,0};
Timer = PeriodicCallback(0, 5, 10, TIMER, 10, 1,
    1, sequence1, sequence1, 0);
Client = DataCallback(0, 5, 1, CLIENT, 10, 0,
    emptyArrayPub, emptyArrayPub, 0);
Node1 = ExecutorV1(1, StopTime);
const int publishTimes[MAXPUB
    ]={5,0,0,0,0,0,0,0,0,0};
const int publisherIds[MAXPUB
    ]={1,0,0,0,0,0,0,0,0,0};
Service = DataCallback(0, 5, 0, SERVICE, 10, 1,
    publishTimes, publisherIds, 1);
system Node0, Node1, Service, Timer, Client,
    Request, Response;
```

Listing 6 Example of a UPPAAL system declaration for a processing chain (holistic approach)

```
const int StopTime = -1;
Msg1 = Topic(0,0,0,0,10);
Msg2 = Topic(1,1,0,0,10);
const int publishTimes[MAXPUB
    ]={5,0,0,0,0,0,0,0,0,0};
Node0 = ExecutorV2(0, StopTime);
const int N0PublisherIds[MAXPUB
    ]={0,0,0,0,0,0,0,0,0,0};
Timer = PeriodicCallback(0, 5, 10, TIMER, 0, 1, 1,
     publishTimes, N0PublisherIds, 0);
Node1 = ExecutorV1(1, StopTime);
const int N1PublisherIds[MAXPUB
    ]={1,0,0,0,0,0,0,0,0,0};
PubSub = DataCallback(0, 5, 0, SUBSCRIBER, 10, 1,
    publishTimes, N1PublisherIds, 1);
Timer2 = PeriodicCallback(0, 5, 15, TIMER, 5, 1,
    0, publishTimes, N1PublisherIds, 1);
Node2 = ExecutorV1(2, StopTime);
const int N2PublisherIds[MAXPUB
    ]={0,0,0,0,0,0,0,0,0,0};
Sub = DataCallback(0, 5, 0, SUBSCRIBER, 10, 0,
    publishTimes, N2PublisherIds, 2);
system Node0, Node1, Node2, Timer, Timer2, PubSub,
     Sub, Msg1, Msg2;
```

Listing 7 Example of a UPPAAL system declaration for a node in a processing chain (individual node approach)

```
const int StopTime = 50;
const int releases[MAXX
    ]={15,20,30,45,0,0,0,0,0,0};
ExecV1 = ExecutorV1(0,StopTime);
Sub = SporadicCallback(0, 5, 4, releases,
    SUBSCRIBER,10, 0, emptyArrayPub, emptyArrayPub
    , 0);
system ExecV1, Sub;
```

Acknowledgements This work has been supported by the Swedish Knowledge Foundation via the profile DPAC – Dependable Platform for Autonomous Systems and Control, grant nr: 20150022, the synergy ACICS – Assured Cloud Platforms for Industrial Cyber-Physical Systems, grant nr. 20190038, and SEINE – Automatic Self-configuration of Industrial Networks, grant nr: 20220230, and by the Swedish Governmental Agency for Innovation Systems (VINNOVA) via the INTERCONNECT, PROVIDENT, and FLEXATION projects.

Funding information Open access funding provided by Mälardalen University.

References

1. Al-Bataineh, O., Reynolds, M., French, T.: Accelerating worst case execution time analysis of timed automata models with cyclic behaviour. Form. Asp. Comput. **27**, 917–949 (2015)
2. Albonico, M., Dordevic, M., Hamer, E., Malavolta, I.: Software engineering research on the robot operating system: a systematic mapping study. J. Syst. Softw. **197**, Article ID 111574 (2023). https://doi.org/10.1016/j.jss.2022.111574
3. Alur, R., Dill, D.L.: A theory of timed automata. Theor. Comput. Sci. **126** (1994)
4. Anand, A., Knepper, R.: Roscoq: robots powered by constructive reals. In: Urban, C., Zhang, X. (eds.) Interactive Theorem Proving, pp. 34–50. Springer, Cham (2015)
5. Baier, C., Katoen, J.P.: Principles of Model Checking. MIT Press, Cambridge (2008)
6. Blaß, T., Casini, D., Bozhko, S., Brandenburg, B.B.: A ROS 2 response-time analysis exploiting starvation freedom and execution-time variance. In: IEEE Real-Time Systems Symposium, pp. 41–53. IEEE Press, New York (2021)
7. Carvalho, R., Cunha, A., Macedo, N., Santos, A.: Verification of system-wide safety properties of ROS applications. In: 2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS) (2020)
8. Casini, D., Blaß, T., Lütkebohle, I., Brandenburg, B.: Response-time analysis of ROS 2 processing chains under reservation-based scheduling. In: 31st Euromicro Conference on Real-Time Systems, pp. 1–23 (2019)
9. Dalsgaard, A., Olesen, M., Toft, M., Hansen, R., Larsen, K.: METAMOC: modular execution time analysis using model checking. In: Proceedings of the 10th International Workshop on Worst-Case Execution-Time Analysis (WCET2010) (2010)
10. Dust, L., Gu, R., Seceleanu, C., Ekström, M., Mubeen, S.: Pattern-based verification of ROS 2 nodes using UPPAAL. In: FMICS 2023 – International Conference on Formal Methods for Industrial Critical Systems (2023)
11. Halder, R., Proença, J., Macedo, N., Santos, A.: Formal verification of ROS-based robotic applications using timed-automata. In: 2017 IEEE/ACM 5th International FME Workshop on Formal Methods in Software Engineering (FormaliSE), pp. 44–50 (2017)
12. Hendriks, M., Yi, W., Petterson, P., Hakansson, J., Larsen, K., David, A., Behrmann, G.: UPPAAL 4.0. In: Third International Conference on the Quantitative Evaluation of Systems – (QEST'06) (2006)
13. OpenRobotics: Ros: Distributions (2023). http://wiki.ros.org/Distributions
14. OpenRobotics: Ros 2: Distributions (2023). https://docs.ros.org/en/humble/Releases
15. OpenRobotics: Ros 2: Documentation (2023). https://docs.ros.org/en/humble
16. Quigley, M., Conley, K., Gerkey, B., Faust, J., Foote, T., Leibs, J., Wheeler, R., Ng, A.Y., et al.: ROS: an open-source robot operating system. In: ICRA Workshop on Open Source Software, vol. 3, pp. 5. Kobe, Japan (2009)
17. Rajkumar, R., Lee, I., Sha, L., Stankovic, J.: Cyber-physical systems: the next computing revolution. In: Proceedings of the 47th Design Automation Conference, pp. 731–736 (2010)
18. Tang, Y., Feng, Z., Guan, N., Jiang, X., Lv, M., Deng, Q., Yi, W.: Response time analysis and priority assignment of processing chains on ROS2 executors. In: IEEE Real-Time Systems Symposium, pp. 231–243 (2020)
19. Teper, H., Günzel, M., Ueter, N., von der Brüggen, G., Chen, J.J.: End-to-end timing analysis in ROS 2. In: 2022 IEEE Real-Time Systems Symposium (RTSS), pp. 53–65 (2022)
20. Webster, M., Dixon, C., Fisher, M., Salem, M., Saunders, J., Koay, K.L., Dautenhahn, K., Saez-Pons, J.: Toward reliable autonomous robotic assistants through formal verification: a case study. IEEE Trans. Human-Mach. Syst. **46**(2), 186–196 (2016)